

Lastminute AI Slop Vorbereitung

Basierend auf den „geleakten“ Themen und den bereitgestellten Unterlagen habe ich dir hier eine **Roadmap für die Last-Minute-Wiederholung** zusammengestellt. Diese Punkte decken die theoretischen Konzepte und typischen Prüfungsfragen ab.

1. Deadlock Avoidance (Banker's Algorithm)

Hier musst du wissen, wie man entscheidet, ob ein Systemzustand **sicher (safe)** oder **unsicher (unsafe)** ist und ob eine Ressourcenanforderung gewährt werden darf.

- **Das Prinzip:** Ressourcen werden nur gewährt, wenn der resultierende Zustand **safe** ist. Ein Zustand ist **safe**, wenn es eine Reihenfolge gibt, in der alle Prozesse beendet werden können.
- **Vorgehensweise (Schnellcheck):**
 1. Berechne die Matrix $\text{Need} (\text{Max} - \text{Allocation})$.
 2. Vergleiche Need eines Prozesses mit den verfügbaren Ressourcen ($\text{Available} / \text{Work}$).
 3. Finde einen Prozess, dessen Bedarf gedeckt werden kann ($\text{Need} \leq \text{Work}$).
 4. Simuliere dessen Ende: Addiere seine Allocation zu Work ($\text{Work} = \text{Work} + \text{Allocation}$).
 5. Wiederhole das, bis alle Prozesse fertig sind (Safe) oder keiner mehr ausführbar ist (Unsafe).
- **Wichtig:** Ein **Unsafe State** bedeutet nicht zwingend sofortigen Deadlock, aber ein Deadlock ist möglich.

2. Semaphore (Parallel / max_two)

Das bezieht sich auf Synchronisation, insbesondere wenn mehr als einer, aber maximal N Prozesse zugreifen dürfen (z. B. 2 Schiffe auf einer Brücke).

- **Grundoperationen:**
 - $P(S)$ (Wait): Dekrementiert S. Wenn $S < 0$, blockiert der Prozess.
 - $V(S)$ (Signal): Inkrementiert S. Weckt ggf. wartenden Prozess auf.
- **Szenario "max_two":**
 - Initialisiere das Semaphor mit dem Wert 2 ($\text{init}(S, 2)$).
 - Die ersten zwei Prozesse, die $P(S)$ rufen, dürfen passieren (S wird 1, dann 0). Der dritte blockiert (S wird -1),.
- **Reihenfolge:** Achte bei Aufgaben darauf, dass Semaphoren für **Mutual Exclusion** (z. B. Zugriff auf Variablen) meist mit 1 initialisiert werden, während solche für **Kapazitätsbegrenzungen** (wie 2 Schiffe) mit N initialisiert werden.

3. Puffer (Producer-Consumer / 1/0)

Dies ist das klassische Erzeuger-Verbraucher-Problem, oft mit einem Ringpuffer.

- **Benötigte Semaphoren:**
 - S (Mutex): Schützt den kritischen Abschnitt (Zugriff auf Puffer-Indizes). Init = 1.
 - N (Items): Zählt volle Plätze. Init = 0 (für den Consumer).

- **E** (Empty): Zählt freie Plätze. Init = Puffergröße K (für den Producer).
- **Der Code (Ablauf):**
 - **Producer:** `produce(v)` -> `P(E)` (Ist Platz?) -> `P(S)` (Sperre Puffer) -> `append(v)` -> `V(S)` -> `V(N)` (Sag Consumer Bescheid).
 - **Consumer:** `P(N)` (Ist was da?) -> `P(S)` -> `take()` -> `V(S)` -> `V(E)` (Platz frei geworden).
- **Wichtig:** Niemals `P(S)` *vor* `P(E)` oder `P(N)` machen, sonst droht Deadlock (wenn Puffer voll/leer ist und der Prozess den Mutex hält, aber auf Platz/Daten wartet).

4. Virtual Memory Management

Hier geht es um die Mechanismen, die den logischen vom physischen Speicher trennen.

- **Konzept:** Logische Adressen werden zur Laufzeit übersetzt. Ein Prozess muss nicht vollständig im RAM sein (Resident Set), sondern nur die benötigten *Pages*,.
- **Mechanismen (Was wird benötigt?):**
 - **Paging:** Unterteilung in Pages (logisch) und Frames (physisch) gleicher Größe.
 - **Page Table:** Speichert Zuordnung Page -> Frame und Bits (Present, Modified).
 - **TLB (Translation Lookaside Buffer):** Hardware-Cache für schnelle Adressübersetzung, vermeidet doppelten Speicherzugriff.
- **Vorteil:** Prozesse können größer als der physische Hauptspeicher sein; höherer Grad an Multiprogramming.

5. Security Threat (Nenne 3)

Die klassischen Bedrohungskategorien (Threats) richten sich gegen die CIA-Ziele (Confidentiality, Integrity, Availability).

1. **Denial of Service (Interruption):** Angriff auf die *Availability* (Verfügbarkeit). Zerstörung oder Überlastung eines Dienstes,.
2. **Exposure / Interception:** Angriff auf die *Confidentiality* (Vertraulichkeit). Unautorisierter Lesezugriff (Abhören),.
3. **Modification / Fabrication:** Angriff auf die *Integrity* (Integrität). Unautorisiertes Verändern oder Erzeugen von Daten,.

6. Datei Vor/Nachteile (Allokierung)

Wie werden Dateiblöcke auf der Festplatte organisiert?

- **Contiguous Allocation (Zusammenhängend):**
 - **Vorteil:** Sehr schnell beim Lesen (Sequential & Random).
 - **Nachteil:** Externe Fragmentierung; Datei kann nicht wachsen, wenn kein Platz dahinter frei ist.
- **Chained Allocation (Verkettet):**
 - **Vorteil:** Keine externe Fragmentierung, Datei kann leicht wachsen.
 - **Nachteil:** Schlechter Random Access (man muss der Kette folgen); Platzverlust im Block durch Pointer.

- **Indexed Allocation (z. B. I-Nodes):**
 - *Vorteil:* Guter Random Access; keine externe Fragmentierung.
 - *Nachteil:* Speicheroverhead für Index-Blöcke/Tabellen.

7. Feedback Scheduling & Scheduling Schemes

Hier geht es um die Strategien, wie die CPU vergeben wird.

- **Feedback Scheduling:**
 - *Prinzip:* Preemptive. Priorität basiert auf bisheriger CPU-Nutzung. Prozesse starten mit hoher Priorität. Wenn sie viel CPU brauchen, sinken sie in niedrigere Queues ab (Bestrafung CPU-intensiver Prozesse).
 - *Problem:* Starvation möglich -> Lösung: Priorität nach Wartezeit wieder anheben (Aging).
- **Andere Schemes (Kurzvergleich):**
 - **FCFS (First Come First Served):** Simpel, aber "Convoy Effect" (kurze warten hinter langen).
 - **RR (Round Robin):** Fair, feste Zeitscheibe (Quantum), preemptive. Gut für Response Time, schlecht für I/O-Prozesse (daher Virtual Round Robin nutzen),.
 - **SPN (Shortest Process Next):** Minimiert Wartezeit, aber Starvation langer Prozesse möglich. Erfordert Kenntnis der Laufzeit.
 - **HRRN (Highest Response Ratio Next):** Berücksichtigt Wartezeit $(w+s)/s$. Verhindert Starvation, ist aber non-preemptive.